Universidad Politecnica de Madrid

Explainable Classification of Payment Difficulty

Phases I–II: Decision Tree and Logistic Regression

Group members: Thales S. A. Camargo – thales.camargo@alumnos.upm.es
Jhon Corrales Almendariz – jhon.corrales@alumnos.upm.es
Joel Montes de Oca Martínez – joel.montesdeoca.martinez@alumnos.upm.es

Course: Explainable Artificial Intelligence
Programme: Master in Artificial Intelligence
Date: September 2026

Preliminary report for Tema 2 practical work

Abstract

This report documents the decision-tree and logistic-regression phases of the Tema 2 practical work in Explainable Artificial Intelligence. Across five stratified folds, the default logistic model achieves 0.788 +/- 0.000 accuracy but only 0.501 +/- 0.001 balanced accuracy, with 0.002 +/- 0.002 recall for the difficult-payment class. Hyperparameter tuning with balanced class weights raises its hold-out balanced accuracy to 0.680, at 0.713 accuracy. On the same validation split, the optimized tree obtains 0.609 balanced accuracy and 0.813 accuracy. The comparison illustrates the trade-off between overall accuracy and balanced detection of both classes.

1 Introduction

The task is a binary classification problem based on customer credit and repayment information. The training data contain the target variable `payment_difficulty_next_month`; the test data contain the same predictors without that target. The objective is both predictive and explanatory: the model should produce test predictions while allowing its behavior to be inspected.

The decision-tree and logistic-regression implementations are provided in the Phase I and Phase II notebooks under `code/`. Both use a reproducible preprocessing pipeline, hold-out validation, and model-specific prediction exports, while their model families and explanation methods remain distinct. The default models favor the majority class; balanced-class hyperparameter tuning improves balanced accuracy, though the logistic model does so at the cost of lower overall accuracy.

The tuned decision tree predicts class 1 for only 4.1% of test observations. Balanced accuracy and class-specific recall are therefore essential evidence rather than presenting accuracy in isolation. The logistic model's higher hold-out balanced accuracy is promising, but it should be considered alongside its lower accuracy and its class-specific errors.

2 Data and preprocessing

The notebook defines the feature roles explicitly rather than selecting columns implicitly. The identifier and target are excluded from the predictor matrix, while demographic, credit-limit, billing, payment, and repayment variables are used as predictors. Sex is retained as an audit attribute as well as a predictor, which makes later fairness-oriented analysis possible.

The categorical variables are sex, education, marital status, and repayment assistance plan. They are transformed with one-hot encoding using `handle_unknown="ignore"`, so categories that appear only in the test split do not cause prediction failures. Numerical variables are concatenated with the encoded categorical variables without additional scaling.

The labeled training data are split into training and validation partitions with a 70/30 ratio, a fixed random seed of 411, and stratification by the target. The unlabeled test data remain untouched until the final prediction stage. This procedure gives a reproducible validation estimate while preserving the test set for submission.

3 Method

The implementation uses pandas for tabular data loading and manipulation, scikit-learn for the stratified split, one-hot encoding, logistic-regression parameter search, and evaluation metrics, and InterpretML for the class histogram, classification tree, logistic regression, and model explanations. Matplotlib is used for the feature-importance and predicted-class proportion plots. Shared data preparation and prediction-export functions are defined in `code/common.py`; dependencies are listed in `code/requirements.txt`.

Only the labeled training data are split. Seventy percent is used for fitting and thirty percent for validation, with stratification preserving the class proportions and `random_state=411` making the result reproducible. The separate test file is never used to fit the encoder or model: its categorical values are transformed after the encoder has been fitted on the training partition, with unknown categories ignored.

The notebooks use five-fold stratified cross-validation to characterize the default models and select hyperparameters using balanced accuracy. For the tree, the selected depth is 5 and the minimum leaf size is 50, with a best search score of 0.690. For logistic regression, the selected settings are $C = 100$ and balanced class weights, with a best search score of 0.671. Each final model is evaluated separately on the same 30% hold-out partition. A grid-search score is a parameter-selection result, not an independent estimate of the final refit. The separate test file is not used in model selection or validation. The Explainable Boosting Machine remains to be added for the complete three-family comparison.

4 Results

4.1 Decision Tree

Table 1 summarizes the selected tree configuration and its best grid-search score. Table 2 reports the separate hold-out results for the refitted optimized tree. The default-tree cross-validation metrics in the abstract provide a baseline and should not be confused with the tuned model's hold-out results.

Table 1: Phase I model summary.

Classifier	Key settings	Five-fold CV result(s)
ClassificationTree	max_depth=5, min_samples_leaf=50; unweighted refit	Search balanced accuracy 0.690*

*The score is from the class-weighted scikit-learn tree used for parameter search; it is not an independent cross-validation estimate of the final InterpretML refit.

Table 2: Decision-tree performance on the 30% stratified validation split.

Metric	Value
Accuracy	0.813
Balanced accuracy	0.609
Class 0 precision / recall / F1	0.83 / 0.96 / 0.89
Class 1 precision / recall / F1	0.65 / 0.26 / 0.37
Validation observations	7,500

The original hold-out validation set contains 5,909 class-0 and 1,591 class-1 observations. The optimized tree improves class-1 recall to 0.26, increasing balanced accuracy to 0.609. The gap between class recalls explains why balanced accuracy is still lower than accuracy: the tree identifies most non-difficult cases but misses many difficult-payment cases. On the unlabeled test set, we anticipate a slightly better detection of class 1 due to the hyperparameter optimization.

The optimized tree's global explanation is generated with `tree.explain_global`. Figure 1 complements the interactive explanation with the fitted tree's split-based feature-importance ranking. Repayment status and the payment-to-bill ratio dominate the model's splits; this ranking describes the fitted tree and should not be read as a causal effect.

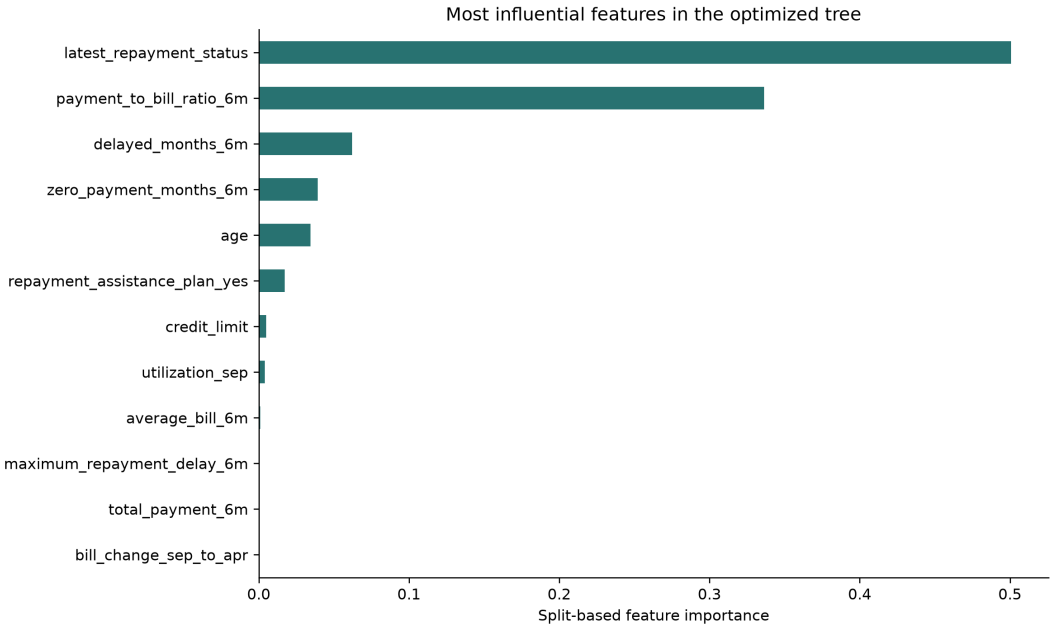


Figure 1: The twelve highest split-based feature importances for the optimized tree.

4.2 Logistic Regression

Table 3 summarizes the default and tuned logistic models. The default model nearly always predicts the majority class: its mean class-1 recall is 0.002 across the five folds, despite accuracy of 0.788. The grid search selects $C = 100$ and balanced class weights. On the hold-out split, tuning raises balanced accuracy from 0.501 to 0.680, while accuracy falls from 0.788 to 0.713. This is a more balanced result than the optimized tree's 0.609 balanced accuracy, but the tree retains higher accuracy at 0.813. The two metrics therefore describe a genuine trade-off rather than one model dominating on every criterion.

Table 3: Logistic-regression cross-validation and hold-out results.

Model	Five-fold CV balanced accuracy	Hold-out accuracy	Hold-out balanced accuracy
Default InterpretML logistic regression	0.501 ± 0.001	0.788	0.501
Tuned logistic regression ($C = 100$, balanced weights)	0.671*	0.713	0.680

*The score is the best GridSearchCV selection score; it is not an independent estimate of the final refit.

The optimized model's coefficient-magnitude ranking (Figure 2) places latest repayment status, zero-payment months, and delayed months among the strongest features. Repayment-assistance indicators and utilization also appear in the top twelve. The plot shows absolute coefficient magnitudes, so it indicates relative influence in this fitted linear model but not the direction of each association or a causal effect.

The notebook also examines representative local validation cases. It includes a correctly detected class-1 case with predicted probability 0.656 and a missed class-1 case with probability 0.377, below the model's 0.5 decision threshold. These examples make the probability-to-class decision visible, but they are illustrative cases rather than estimates of overall error rates.

5 Models

The classifier is `ClassificationTree` from the `interpret` package. A decision tree is suitable for this phase because its decision structure can be inspected directly and because the package exposes global explanations of the fitted model [Nori et al., 2019]. We used `GridSearchCV` to optimize hyperparameters, selecting a `max_depth` of 5 and `min_samples_leaf` of 50 based on balanced accuracy. The notebook also displays a class histogram for the training data, a confusion matrix, and a ROC curve with AUC metric.

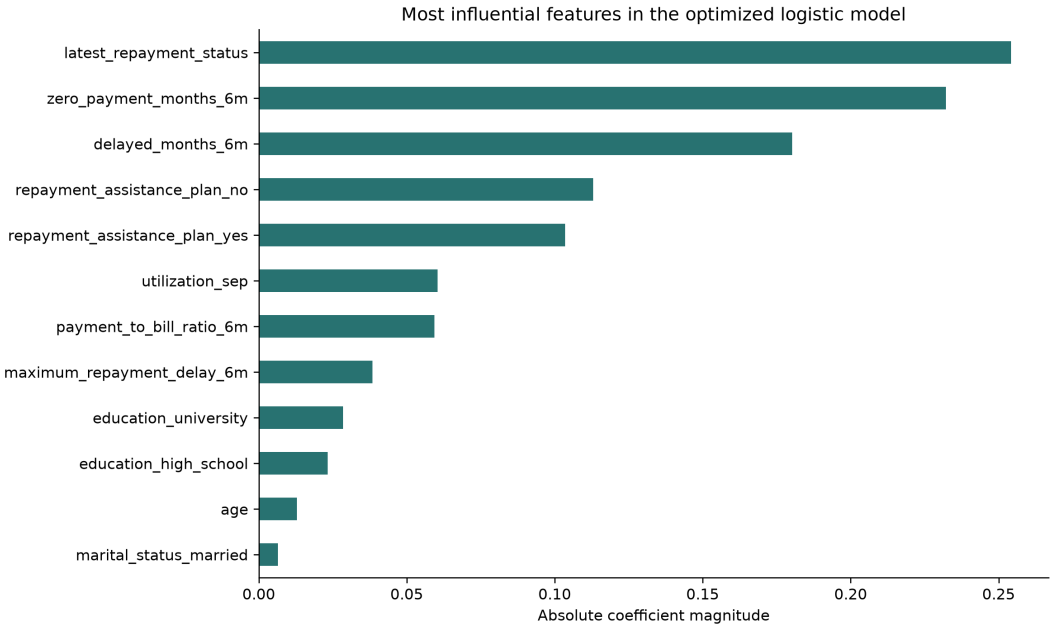


Figure 2: The twelve largest absolute coefficients in the optimized logistic model.

The tree is transparent in principle: each prediction can be traced through a sequence of feature conditions, and the global explanation exposes the learned structure. Logistic regression offers a different form of transparency: its prediction is an additive function of the encoded features, and the coefficient magnitudes identify which terms most affect the fitted score. Neither representation should be interpreted causally, and the different explanation formats are not directly interchangeable.

The optimized tree's global explanation is generated with `tree.explain_global`. Its split-based feature-importance ranking places repayment status and the payment-to-bill ratio among the most influential features; this ranking describes the fitted tree and should not be read as a causal effect.

Figure 3 shows the first three levels of the optimized tree. The full depth-five tree is available in the notebook; limiting the report view keeps its main decision structure readable without hiding that lower-level branches exist.

For local interpretation, the notebook selects fixed cases from the validation split, including correctly predicted examples from both classes and a missed class-1 example. Table 4 summarizes their paths. The missed positive reaches a class-0 leaf despite a repayment-assistance indicator, illustrating why the global model's strong majority-class behavior matters for individual decisions.

Table 4: Representative local decision paths from the validation set.

Case	Actual	Predicted	Path summary
Correct class 0	0	0	Non-delayed repayment status; payment-to-bill ratio above 0.105 and at most 1.91; no assistance plan; at most 2.5 zero-payment months.
Detected class 1	1	1	Delayed repayment status; payment-to-bill ratio at most 0.095; at most 5.5 delayed months; more than 2.5 zero-payment months; age above 25.5.
Missed class 1	1	0	Non-delayed repayment status; payment-to-bill ratio between 0.025 and 0.105; age above 24.5; assistance-plan flag is yes.

These are illustrative cases, not a substitute for evaluating the model across the full validation set. The interactive local explanations and exact split conditions for each selected case are included in the notebook.

The remaining model family for the complete project is an Explainable Boosting Machine with zero interactions, which would allow flexible feature-wise effects while retaining global interpretability. Its five-fold results and local explanations should be added before drawing a final comparison across all required model families.

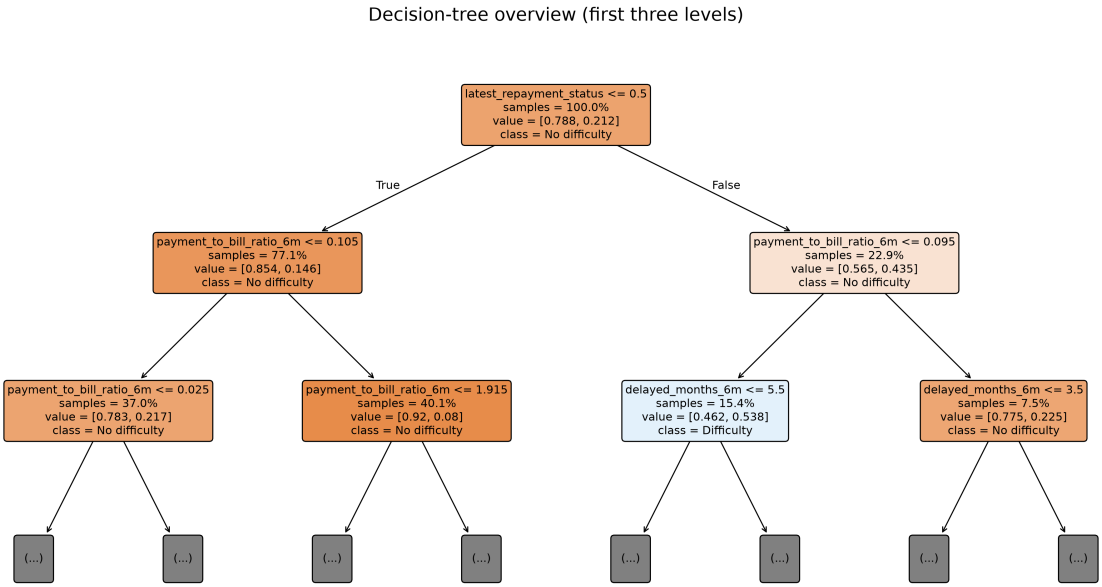


Figure 3: Readable overview of the optimized decision tree. Node colors indicate the predicted class and shading indicates class purity; lower branches are collapsed.

6 Test predictions

Once validation is complete, each fitted model predicts the unlabeled test set. The notebooks preserve the original test predictors, append `predicted_class`, and save separate files: `data/decision_tree_test_predictions.csv` and `data/logistic_regression_test_predictions.csv`. Each also plots the predicted-class proportions. Keeping the original predictors beside the predictions makes the generated files easier to audit.

7 Limitations and future work

The current comparison includes a tree and logistic regression, but its grid-search scores are selection results and should not be treated as independent estimates. Future work should add the Explainable Boosting Machine, compare all three families under the same protocol, and assess threshold choices and class-specific error costs when false negatives and false positives are unequal. The tuned logistic results should also be supplemented with class-specific hold-out metrics when reporting the final model comparison.

The notebook already preserves sex as an audit attribute, but it does not yet calculate group-wise performance or fairness measures. A subsequent phase should compare error rates across relevant groups, inspect feature-importance stability, and use local explanations for the same representative instances across all three model families, including at least one instance from each class. Any such analysis should be performed without using the test labels.

8 Discussion

The tree is attractive because its rules can be communicated directly, while the logistic model provides an additive explanation through its coefficients. On the shared validation split, logistic regression achieves higher balanced accuracy than the tree after class-weighted tuning, but at lower accuracy. This reinforces the need to choose a model using the class-specific costs of errors rather than accuracy alone. Both tuned models improve on their majority-class-biased defaults, although the logistic model's default class-1 recall is almost zero.

Interpretability also has a cost. A fully expanded tree can become a long and repetitive list of rules, making individual paths technically transparent but difficult to understand. Limiting the depth to 5 not only improved the balanced accuracy but also restricted the tree size, making the global and local explanations much more understandable. The same trade-off should be assessed for logistic regression coefficients and additive feature-effect plots rather than assuming that one explanation format is universally superior.

9 Conclusion

The decision-tree and logistic-regression phases establish a reproducible pipeline from raw credit and repayment data to interpretable models and separate test prediction files. On the hold-out split, the tuned tree reaches 0.813 accuracy and 0.609 balanced accuracy, while tuned logistic regression reaches 0.713 accuracy and 0.680 balanced accuracy. The tree is more accurate overall; the logistic model is more balanced across classes. Adding the Explainable Boosting Machine and considering the cost of missed payment-difficulty cases are necessary before selecting a final model.

10 Use of generative AI

Generative AI was used to help organize the report structure, improve wording, and identify gaps between the Phase I notebook and the project statement. All numerical results and methodological claims in this report were checked against the repository notebook and its recorded outputs. The group remains responsible for the code, analysis, interpretation, and final submission.

References

Harsha Nori, Samuel Jenkins, Paul Koch, and Rich Caruana. Interpretml: A unified framework for machine learning interpretability. In *Proceedings of the 2019 Workshop on Human-Centric Machine Learning at NeurIPS*, 2019. URL https://github.com/interpretml/interpret.